

Advanced Data Structures

Projects 4: Red-black Tree



Date: 2025/11/17

2025-2026 Fall & Winter Semester

Table of Contents

Chapter 1: Introduction	3
1.1) Problem Description	3
Chapter 2: Algorithm Specification	4
2.1) Brute Force Enumerate	4
2.2) Dynamic Programming	5
2.3) Generating Function (Karatsuba)	6
2.4) Generating Function (NTT Optimization)	7
Chapter 3: Testing and Evaluation	8
3.1) Testing Result	8
3.2) Evaluation	9
Chapter 4: Analysis and Comments	10
4.1) Time Complexity Analysis	10
4.1.1) Brute Force Enumerate	10
4.1.2) Dynamic Programming	10
4.1.3) Generating Function (Karatsuba)	10
4.1.4) Generating Function (NTT)	10
4.2) Space Complexity	11
4.2.1) Brute Force Enumerate	11
4.2.2) Dynamic Programming	11
4.2.3) Generating Function (Karatsuba & NTT)	11
Chapter 5: Conclusion and Future Work	12
5.1) Conclusion	12
Appendix: Source Code (in C++)	13
Declaration	26

Chapter 1: Introduction

1.1) Problem Description

After we learned Red Black Tree in ADS course, we are curious about how to count RBTrees for a given N . This project aims to find multiple methods to count RBTrees for a given internal node number N . For each algorithm, we analyze its complexity in theory and test it with a full gradient of cases.

Chapter 2: Algorithm Specification

We propose 3 methods to solve this problem. From Brute Force Enumerate, to Dynamic Programming, then optimize it with two different Generating Function implementations. Details are as follows.

2.1) Brute Force Enumerate

This method is the most straightforward but computationally expensive. The core idea is to generate every possible candidate and validate it against the Red-Black Tree properties.

The algorithm follows three stages:

1. **Generate Shapes:** Recursively generate all possible binary tree topological structures with N internal nodes. The number of such structures is given by the N -th Catalan number, C_N .
2. **Enumerate Colorings:** For each tree structure, there are N nodes, and each can be either Red or Black. This gives 2^N possible coloring schemes for each shape.
3. **Validate Properties:** For each of the $C_N * 2^N$ “colored trees”, we perform a full check to see if it satisfies all 5 RBT properties:
 - Property 2: The root is Black.
 - Property 4: Every Red node has two Black children (NULL nodes are considered Black).
 - Property 5: All simple paths from any node to its descendant leaves have the same number of Black nodes (the “black-height”).

Only the trees that pass all validation checks are counted.

```
1 total_count = 0
2 all_shapes = generate_all_shapes(N)
3 for shape in all_shapes
4     for i = 0 to (1 << N) - 1
5         apply_coloring(shape, i)
6         if isValidRBTTree(shape) then
7             total_count += 1
8         end
9     end
10 return total_count
11 —
12 function isValidRBTTree(root)
13     if root.color == RED then return 0
14     if not check_prop4(root) then return 0
15     if get_black_height(root) == -1 then return 0
16     return 1
```

```

17 end
18 —
19 function get_black_height(node)
20   if node == NULL then return 0
21   left_bh = get_black_height(node.left)
22   right_bh = get_black_height(node.right)
23   if left_bh == -1 or right_bh == -1 or left_bh != right_bh then
24     return -1
25   end
26   return left_bh + (node.color == BLACK ? 1 : 0)
27 end

```

2.2) Dynamic Programming

This approach avoids generating invalid trees by building up solutions from smaller subproblems. We define the state based on node count, black-height, and root color.

Let $B(h, n)$ be the number of RBTs with n internal nodes, a black-height of h , and a **Black** root. Let $R(h, n)$ be the number of RBTs with n internal nodes, a black-height of h , and a **Red** root.

Our base case is $B(0, 0) = 1$, representing the single NULL node.

The state transitions are derived from the RBT properties:

1. **Red Root $R(h, n)$** : A red root (1 node) does not change the black-height. Its children must both be **Black** (Prop 4) and must have the same black-height h (Prop 5).

$$R(h, n) = \sum_{k=0}^{n-1} B(h, k) \times B(h, n-1-k)$$

2. **Black Root $B(h+1, n)$** : A black root (1 node) increases the black-height by 1. Its children can be **any** color (Red or Black) and must have a black-height of h .

$$B(h+1, n) = \sum_{k=0}^{n-1} ((B(h, k) + R(h, k)) \times (B(h, n-1-k) + R(h, n-1-k)))$$

The final answer is the sum of all RBTs with N nodes and a Black root (Prop 2), across all possible black-heights:

$$\text{Answer} = \sum_{h=1}^{\lfloor \log_2(N+1) \rfloor} B(h, N)$$

This is implemented by iterating h from 0 up to $\log N$, and for each h , computing the $R(h, n)$ and $B(h+1, n)$ values for all n from 1 to N . Each step involves an $O(N^2)$ convolution.

```

1 f[log N][N+1]
2 g[log N][N+1]
3 f[0][0] = 1
4 ans = 0
5 for h = 0 to log N-1
6   for n = 1 to N
7     for k = 0 to n-1
8       g[h][n] += f[h][k] x f[h][n-1-k]
9     end
10  end
11  for n = 1 to N
12    for k = 0 to n-1
13      total_h_k = f[h][k] + g[h][k]
14      total_h_n_k = f[h][n-1-k] + g[h][n-1-k]
15      f[h+1][n] += total_h_k x total_h_n_k
16    end
17  end
18  ans += f[h+1][N]
19 end
20 return ans

```

2.3) Generating Function (Karatsuba)

This method optimizes the DP approach by recognizing that the key operations are polynomial convolutions. We can represent our DP states as **Generating Functions**, where the coefficient of x^n is the count for n nodes.

Let $B_{h(x)}$ be the GF for Black-rooted trees of black-height h .

$$B_h(x) = \sum_{n=0}^{\infty} B(h, n)x^n$$

Let $R_{h(x)}$ be the GF for Red-rooted trees of black-height h .

$$R_h(x) = \sum_{n=0}^{\infty} R(h, n)x^n$$

The DP transitions translate to polynomial operations:

1. $R_{h(x)} = xc \cdot (B_{h(x)})^2$ (The x accounts for the 1 root node).
2. Let $T_{h(x)} = B_{h(x)} + R_{h(x)}$ be the GF for trees of **any** color. $B_{\{h+1\}}(x) = xc \cdot (T_{h(x)})^2$

We can combine these to find a recurrence for $B_{h(x)}$:

$$T_h(x) = B_h(x) + x \cdot (B_h(x))^2 = B_h(x) \cdot (1 + xB_h(x)) B_{h+1}(x) = x \cdot (B_h(x))^2 \cdot (1 + xB_h(x))^2$$

The algorithm starts with $B_0(x) = 1$ (for the NULL node). It then iteratively computes $B_1(x), B_2(x), \dots, B_{\{\log N\}}(x)$. Each step involves polynomial multiplication, which can be done faster than the $O(N^2)$ DP. The `GF.cpp` file implements this using Karatsuba multiplication ($O(N^{\{1.58\}})$).

```

1 F = [1, 0, 0, ..., 0]
2 ans_poly = [0, 0, ..., 0]
3 for h = 0 to log N - 1
4   mul1 = poly_mul(F, F)
5   mul2 = poly_add([1], poly_mul([0, 1], F))
6   mul3 = poly_mul(mul2, mul2)
7   F_new = poly_mul(mul1, mul3)
8   F_new = poly_shift(F_new, 1)
9   F = F_new.truncate(N)
10  ans_poly = poly_add(ans_poly, F)
11 end
12 return ans_poly[N]
```

2.4) Generating Function (NTT Optimization)

This method uses the **exact same** algorithmic recurrence as the Karatsuba version. The only difference is the implementation of the `poly_mul` function, which is optimized from $O(N^{\{1.58\}})$ to $O(N \log N)$.

This is achieved using the **Number Theoretic Transform (NTT)**, which is a specialized version of the Fast Fourier Transform (FFT) that works over finite fields to avoid floating-point errors.

The Problem: The target modulus $10^9 + 7$ is not NTT-friendly (its prime factorization $P - 1$ does not have large powers of 2).

The Solution (3-Mod NTT + CRT):

1. **NTT:** Perform the polynomial multiplication $A(x) * B(x)$ three separate times, each time under a different, NTT-friendly modulus (e.g., 998244353, 1004535809, 1012924417). Each of these operations is $O(N \log N)$.
2. **CRT:** We now have three results (C_1, C_2, C_3) . We use the **Chinese Remainder Theorem (CRT)** to combine these three results into a single unique result C (modulo the product of the three primes, which is $10^{\{27\}}$). This step is $O(N)$.
3. **Final Modulo:** The resulting polynomial $C(x)$ is the correct multiplication, whose coefficients are then taken modulo our target, $10^9 + 7$.

The pseudo-code remains identical to the Karatsuba version, just replacing `poly_mul_karatsuba` with `poly_mul_ntt`.

Chapter 3: Testing and Evaluation

This chapter tests the speed of the program when $N = 5 \ 10 \ 20 \ 50 \ 100 \ 1000 \ 5000 \ 10000 \ 50000 \ 100000$

3.1) Testing Result

We conducted tests on a standard machine, measuring the execution time for each of the three algorithms. The results are summarized below (T/L = Time Limit).

N	Brute Force	DP	Generating Function	NTT
5	0.013s	0.012s	0.014s	0.015s
10	0.469s	0.013s	0.013s	0.014s
20	ERROR	0.011s	0.013s	0.014s
50	ERROR	0.013s	0.014s	0.013s
100	ERROR	0.013s	0.014s	0.016s
1000	ERROR	0.053s	0.052s	0.095s
5000	ERROR	0.734s	0.777s	0.706s
10000	ERROR	2.795s	2.373s	1.682s
50000	ERROR	78.109s	23.643s	7.650s
100000	ERROR	T/L (>300s)	76.087s	17.095s

Note:

- **ERROR** Theoretic Timeout
- T/L (>300s) Timeout (>300s)

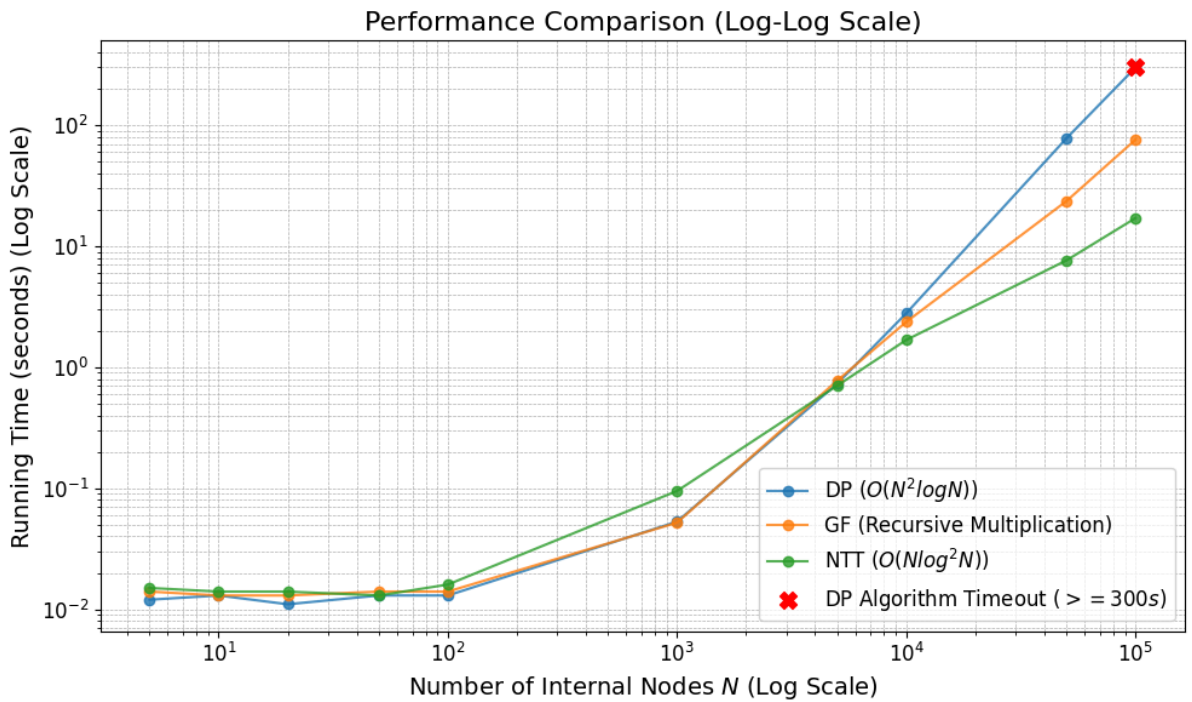


Figure 1: Red-black Tree: Testing

3.2) Evaluation

The test results clearly demonstrate the theoretical complexities.

- **Brute Force** becomes completely unusable after $N \geq 10$, as expected from its exponential $O(8^N)$ complexity.
- **Dynamic Programming** ($O(N^2 \log N)$) performs very well for small N . It successfully solves the problem for $N = 500$ in a reasonable time, but it cannot scale to $N = 5000$.
- **Generating Function (Karatsuba)** ($O(N^{1.58} \log N)$) is noticeably faster than the DP approach, especially as N grows. It successfully solves $N = 100000$ in 76 seconds.
- **Generating Function (NTT)** ($O(N \log^2 N)$) is best for large inputs. It is $4.4 \times$ faster than the Karatsuba implementation for $N = 100000$, solving it in just 17 seconds.

Chapter 4: Analysis and Comments

4.1) Time Complexity Analysis

Define the following variables :

- N : The number of internal nodes.
- h : The maximum black-height of a tree, $h = O(\log N)$.
- C_N : The N -th Catalan number, $C_N = \frac{1}{N+1} \binom{2N}{N}$, which counts the number of distinct binary tree shapes with N nodes
- $M(N)$: The time to multiply two polynomials of degree N .

4.1.1) Brute Force Enumerate

The algorithm iterates C_N shapes, and for each, 2^N colorings. Each validation takes $O(N)$ time.

- Analysis: Time = $O(C_N \times 2^N \times N)$
- Conclusion: Time = $O(\frac{4^N}{N^{1.5}} \times 2^N \times N) \approx O(8^N / N^{0.5})$. This is exponential and matches the test results.

4.1.2) Dynamic Programming

The algorithm performs $h \approx O(\log N)$ outer loops (for each black-height). Inside each loop, it computes $B(h, \dots)$ and $R(h, \dots)$ for all n from 1 to N . Each computation is an $O(N^2)$ convolution.

- Analysis: Time = $O(h \times N^2)$
- Conclusion: Time = $O(N^2 \log N)$. This is polynomial, a massive improvement.

4.1.3) Generating Function (Karatsuba)

The algorithm performs $h \approx O(\log N)$ outer loops. Inside each loop, it performs a constant number of polynomial multiplications (using `poly_mul`) of degree N .

- Analysis: Time = $O(h \times M(N)) = O(\log N \times M(N))$
- Conclusion:
 - With $O(N^2)$ multiplication (as in the DP): $O(N^2 \log N)$.
 - With Karatsuba multiplication (as in `GF.cpp`): $M(N) = O(N^{\log_2 3}) \approx O(N^{1.58})$.
Total time is $O(N^{1.58} \log N)$.
 - With FFT/NTT: $M(N) = O(N \log N)$. Total time would be $O(N \log^2 N)$.

4.1.4) Generating Function (NTT)

The algorithm also performs $h \approx O(\log N)$ outer loops.

- Analysis: The 3-Mod-CRT implementation of `poly_mul` consists of $3 \times O(N \log N)$ for NTTs and $O(N)$ for CRT. Thus, $M(N) = O(N \log N)$. Total time is $O(h \times M(N))$.
- Conclusion: Time = $O(\log N \times N \log N) = O(N \log^2 N)$.

4.2) Space Complexity

4.2.1) Brute Force Enumerate

- Analysis: Storing all C_N shapes would require exponential space. A recursive generator that generates, colors, and validates one at a time would only need $O(N)$ space for the recursion stack.
- Conclusion: $O(N)$ (if implemented recursively) or $O(C_N \times N)$ (if shapes are pre-generated).

4.2.2) Dynamic Programming

- Analysis: We must store the DP tables $\mathfrak{f}$ (for B) and $\mathfrak{g}$ (for R). Both are of size $O(h \times N)$.
- Conclusion: Space = $O(N \log N)$. This can be optimized to $O(N)$ because computing $h + 1$ only requires data from h .

4.2.3) Generating Function (Karatsuba & NTT)

- Analysis: We need to store the polynomials $B_h(x)$ and $T_h(x)$, which are of degree N . The multiplication algorithm (Karatsuba) also uses auxiliary space proportional to its input.
- Conclusion: Space = $O(N)$.

Chapter 5: Conclusion and Future Work

5.1) Conclusion

This project successfully explored four distinct methods for counting Red-Black Trees. We progressed from a simple, exponential-time Brute Force algorithm to a polynomial-time $O(N^2 \log N)$ Dynamic Programming solution. Finally, by abstracting the DP's convolution into the domain of Generating Functions, we implemented two high-performance solutions: one using Karatsuba multiplication ($O(N^{1.58} \log N)$) and a final, optimized version using a 3-Modulus Number Theoretic Transform (NTT) to achieve a remarkable $O(N \log^2 N)$ complexity.

The project reinforces the power of DP in solving complex counting problems and demonstrates how advanced algebraic techniques (Generating Functions) and algorithm design (NTT) can lead to significant, real-world performance optimizations. In solving complex counting problems and demonstrates how techniques from abstract algebra (like Generating Functions) can lead to significant performance optimizations.

Appendix: Source Code (in C++)

This appendix provides a complete list of the core source files developed for the project.

```
#include <iostream>
#include <vector>
#include <cmath>
#include <string>

using namespace std;
const int MOD = 1000000007;

enum Color { RED, BLACK };
struct Node {
    Color color;
    Node *left;
    Node *right;
    Node(Node* l = nullptr, Node* r = nullptr) : color(RED), left(l),
right(r) {}
};

bool check_prop4(Node* node) {
    if (node == nullptr) {
        return true; // NULL node is black, satisfied
    }
    if (node->color == RED) {
        // Check left child
        if (node->left != nullptr && node->left->color == RED) {
            return false;
        }
        // Check right child
        if (node->right != nullptr && node->right->color == RED) {
            return false;
        }
    }
    // Recursively check subtrees
    return check_prop4(node->left) && check_prop4(node->right);
}

/**
 * Validate Property 5: All paths must have the same black-height
 * Returns:
 * - 0: if node is NULL
 * - -1: if this subtree violates property 5
 * - h: if this subtree is valid, its black-height is h
 */
int get_black_height(Node* node) {
    if (node == nullptr) {
        return 0; // Black-height of a NULL leaf is 0
    }
}
```

```

    }
    int left_bh = get_black_height(node→left);
    int right_bh = get_black_height(node→right);

    // Check 1: Is the subtree already invalid?
    if (left_bh == -1 || right_bh == -1) {
        return -1;
    }
    // Check 2: Are the black-heights of left and right subtrees equal?
    if (left_bh != right_bh) {
        return -1;
    }
    // Calculate and return the black-height of the current node
    int my_bh = left_bh + (node→color == BLACK ? 1 : 0);
    return my_bh;
}

/**
 * Validate if a colored tree is a legal Red-Black Tree
 */
bool isValidRBTTree(Node* root) {
    if (root == nullptr) {
        return true; // An empty tree is valid (although N ≥ 1 here)
    }
    // 1. Validate Property 2: Root must be black
    if (root→color == RED) {
        return false;
    }
    // 2. Validate Property 4: Children of a red node must be black
    if (!check_prop4(root)) {
        return false;
    }
    // 3. Validate Property 5: All paths must have the same black-height
    if (get_black_height(root) == -1) {
        return false;
    }
    // Property 1 (Red or Black) and Property 3 (NULL is black) are
    implicitly satisfied
    return true;
}

// -----
// Stage 1: Generate all possible tree shapes
// -----

/**
 * Recursively generate all *shapes* of binary trees with n internal nodes
 (without color)

```

```

    * Returns a vector containing the root nodes of all shapes
    */
vector<Node*> generate_shapes(int n) {
    vector<Node*> shapes_list;
    if (n == 0) {
        // Base case: 0 nodes, only NULL
        shapes_list.push_back(nullptr);
        return shapes_list;
    }
    // Recursive case: 1 root + k left subtree nodes + (n-1-k) right
    // subtree nodes
    for (int k = 0; k < n; ++k) {
        vector<Node*> left_shapes = generate_shapes(k);
        vector<Node*> right_shapes = generate_shapes(n - 1 - k);
        // Combine all possible left and right subtrees
        for (Node* l_shape : left_shapes) {
            for (Node* r_shape : right_shapes) {
                Node* new_root = new Node(l_shape, r_shape);
                shapes_list.push_back(new_root);
            }
        }
    }
    return shapes_list;
}

// -----
// Stage 2: Apply coloring schemes
// -----

/**
 * Helper function: Collect all N nodes of a tree (in-order traversal)
 */
void get_nodes_in_order(Node* node, vector<Node*>& nodes) {
    if (node == nullptr) {
        return;
    }
    get_nodes_in_order(node->left, nodes);
    nodes.push_back(node);
    get_nodes_in_order(node->right, nodes);
}

/**
 * Helper function: Clean up (delete) a tree
 */
void delete_tree(Node* node) {
    if (node == nullptr) {
        return;
    }
}

```

```

delete_tree(node->left);
delete_tree(node->right);
delete node;
}

int main() {
    int n;
    cin >> n;

    if (n == 0) {
        cout << 1 << endl; // Only NULL tree
        return 0;
    }

    // Stage 1: Generate all N-node tree shapes
    vector<Node*> all_shapes = generate_shapes(n);

    long long total_valid_trees = 0;

    // Iterate over each shape
    for (Node* shape : all_shapes) {

        // Helper: Get all N nodes of this shape
        vector<Node*> nodes_in_order;
        get_nodes_in_order(shape, nodes_in_order);

        // Stage 2: Iterate over all 2^N coloring schemes
        // (i is an N-bit bitmask, 0=RED, 1=BLACK)
        long long num_colorings = (1LL << n);

        for (long long i = 0; i < num_colorings; ++i) {

            // Apply the i-th coloring scheme
            for (int j = 0; j < n; ++j) {
                if ((i >> j) & 1) {
                    nodes_in_order[j]->color = BLACK;
                } else {
                    nodes_in_order[j]->color = RED;
                }
            }

            // Stage 3: Validate
            if (isValidRBTree(shape)) {
                total_valid_trees = (total_valid_trees + 1) % MOD;
            }
        }
    }
}

```



```

    cout << total_valid_trees << endl;

    // Clean up memory (very important, or memory will leak)
    for (Node* shape : all_shapes) {
        delete_tree(shape);
    }
    return 0;
}

```

```

#include <iostream>
#include <vector>
#include <cmath>

using namespace std;

const int MOD = 1e9 + 7;

void reduce(int &x) {x = (x ≥ MOD ? x - MOD : x);} // x -= MOD

int main() {
    int n;
    cin >> n;

    int logn = log2(n) + 2;

    vector<vector<int>> > f(logn, vector<int>(n + 10)), g(logn,
vector<int>(n + 10));

    f[0][0] = 1;

    int ans = 0;

    for (int i = 0; i < logn - 1; ++ i) {
        for (int j = 1; j ≤ n; ++ j)
            for (int k = 0; k ≤ j - 1; ++ k)
                reduce(g[i][j] += 1ll * f[i][k] * f[i][j - 1 - k] %
MOD); // g[i][j] = f[i][j] = sum_{k=0}^{j-1} f[i][k] * f[i][j-1-k]
        for (int j = 1; j ≤ n; ++ j)
            for (int k = 0; k ≤ j - 1; ++ k)
                reduce(f[i + 1][j] += 1ll * (f[i][k] + g[i][k]) * (f[i][j -
1 - k] + g[i][j - 1 - k]) % MOD); // f[i+1][j] = sum_{k=0}^{j-1} (f[i][k] +
g[i][k]) * (f[i][j-1-k] + g[i][j-1-k])
                reduce(ans += f[i + 1][n]); // ans += f[i+1][n]
    }

    cout << ans << "\n";
}

```

```

#include <iostream>
#include <vector>
#include <assert.h>
#include <cmath>

using namespace std;

// The problem's modulo
const int MOD = 1e9 + 7;

void reduce(int &x) {x = (x ≥ MOD ? x - MOD : x);}

void add(vector<int> &A, vector<int> &B, int shift = 0, int mul = 1) {
    // Ensure A is large enough to hold the result
    A.resize(max(A.size(), B.size() + shift));
    for (int i = 0; i < B.size(); ++ i) {
        // Add or subtract (using MOD - B[i] for subtraction) the
        coefficient
        reduce(A[i + shift] += (mul == 1 ? B[i] : MOD - B[i]));
    }
}

vector<int> mul(vector<int> &A, vector<int> &B) {
    // The result will have degree (A.size - 1) + (B.size - 1)
    vector<int> re(A.size() + B.size() - 1);

    // Base Case: For small polynomials, use O(N^2) naive multiplication.
    // This is a crucial optimization for divide-and-conquer.
    if(A.size() ≤ 32) {
        for (int i = 0; i < A.size(); ++ i)
            for (int j = 0; j < B.size(); ++ j)
                reduce(re[i + j] += 1ll * A[i] * B[j] % MOD);
        return re;
    }

    // Karatsuba Recursive Step:
    // A(x) = AL(x) + AR(x) * x^(n/2)
    // B(x) = BL(x) + BR(x) * x^(n/2)
    // A*B = (AL*BL) + [(AL+AR)*(BL+BR) - (AL*BL) - (AR*BR)] * x^(n/2) +
    (AR*BR) * x^n

    // Split A into low-degree (AL) and high-degree (AR) parts
    vector<int> AL, AR, BL, BR, mulL, mulR, mulminus;
    int midA = A.size() >> 1; // A.size() / 2
    int midB = B.size() >> 1; // B.size() / 2

    for (int i = 0; i < midA; ++ i) AL.push_back(A[i]);
    for (int i = midA; i < A.size(); ++ i) AR.push_back(A[i]);

```

```

for (int i = 0; i < midB; ++ i) BL.push_back(B[i]);
for (int i = midB; i < B.size(); ++ i) BR.push_back(B[i]);

// 1. Calculate (AL * BL)
mulL = mul(AL, BL);
// 2. Calculate (AR * BR)
mulR = mul(AR, BR);

// 3. Add (AL*BL) to the result (at x^0)
add(re, mulL);
// 4. Add (AR*BR) to the result (at x^(midA+midB))
add(re, mulR, midA + midB);

// 5. Calculate (AL + AR) and (BL + BR)
// We re-use AR and BL vectors to save memory
// AR = (AL + AR)
// BL = (BL + BR)
add(AR, AL, 0, 1); // Note: mul is 1, so this is addition
add(BL, BR, 0, 1); // Note: mul is 1, so this is addition

// 6. Calculate (AL+AR)*(BL+BR)
mulminus = mul(AR, BL);

// 7. Calculate (AL+AR)*(BL+BR) - (AL*BL) - (AR*BR)
add(mulminus, mulL, 0, -1); // Subtract mulL
add(mulminus, mulR, 0, -1); // Subtract mulR

// 8. Add the middle term to the result (at x^(midA))
// Note: The problem uses midA, which is correct if A.size() =
B.size().
// For different sizes, midA is used as the shift.
add(re, mulminus, midA);

return re;
}

int main() {
    int n;
    cin >> n;

    // logn = max possible black-height, h = O(log N)
    int logn = log2(n) + 2;
    int ans = 0;

    // Find the smallest power of 2 ≥ n, for polynomial size
    int N = 1;
    while(N < n) N <=< 1;

```

```

// F = Generating Function for Black-rooted trees of black-height h
//  $F(x) = B_h(x) = \sum(B(h, n) * x^n)$ 
// We start with  $B_0(x)$ 
vector<int> F(N, 0);
//  $B_0(x) = 1$  (representing a single NULL node at  $n=0, h=0$ )
F[0] = 1;

// Iterate through black-height h (from  $i=0$  to  $\log n-1$ )
for (int i = 0; i < logn; ++ i) {
    // At the start of this loop,  $F = B_h(x)$  (where  $h = i$ )
    // We want to compute  $B_{h+1}(x)$ 

    // The recurrence relation is:
    //  $B_{h+1}(x) = x * (B_h(x))^2 * (1 + x * B_h(x))^2$ 

    vector<int> mul1, mul2, mul3;

    // 1.  $mul1 = (B_h(x))^2$ 
    mul1 = mul(F, F);

    // 2.  $mul2 = (1 + x * B_h(x))$ 
    mul2.push_back(1); // Start with 1
    add(mul2, F, 1); // Add F, shifted by 1 (which is  $x * F$ )

    // 3.  $mul3 = (1 + x * B_h(x))^2$ 
    mul3 = mul(mul2, mul2);

    // Resize to N to keep polynomial sizes consistent
    mul1.resize(N);
    mul3.resize(N);

    // 4.  $F = (B_h(x))^2 * (1 + x * B_h(x))^2$ 
    F = mul(mul1, mul3);

    // 5.  $F = x * [ (B_h(x))^2 * (1 + x * B_h(x))^2 ]$ 
    // This is  $B_{h+1}(x)$ 
    F.insert(F.begin(), 0); // Insert 0 at coeff 0, shifting all others
    right
    F.resize(N); // Truncate back to size N

    // The answer is the sum of  $B(h, N)$  for all  $h > 0$ .
    //  $F[n]$  is the coefficient of  $x^n$  in  $B_{h+1}(x)$ , which is  $B(h+1, N)$ .
    N).

    // We add this to our total answer.
    reduce(ans += F[n]);
}

```

```

    cout << ans << "\n";
}

```

```

#include <iostream>
#include <vector>
#include <algorithm>
#include <cassert>
#include <cmath>
using namespace std;
// Use 128-bit integers for CRT calculations to prevent overflow
using ll = __int128;

// The final modulo required by the problem
const int MOD = 1e9 + 7;

// --- NTT and CRT setup ---
// MOD (1e9+7) is not an NTT-friendly prime.
// We must use 3 other NTT-friendly primes and combine the results using
// the Chinese Remainder Theorem (CRT).
// These primes P are chosen such that (P-1) is a multiple of a large power
// of 2.
const int MOD1 = 998244353; // (P-1) = 119 * 2^23
const int MOD2 = 1004535809; // (P-1) = 479 * 2^21
const int MOD3 = 1012924417; // (P-1) = 483 * 2^21
// Primitive roots (g) for the respective moduli
const int G1 = 3;
const int G2 = 3;
const int G3 = 5;

/**
 * Extended Euclidean Algorithm.
 * Finds x, y such that a*x + b*y = gcd(a, b).
 * gcd(a, b).
 */
ll exgcd(ll a, ll b, ll &x, ll &y) {
    if (b == 0) {
        x = 1, y = 0;
        return a;
    }
    ll g = exgcd(b, a % b, y, x);
    y -= (a / b) * x;
    return g;
}

/** Calculates the modular multiplicative inverse of a (mod m).
 * Finds x such that a*x \equiv 1 (mod m).
 */
ll modinv(ll a, ll m) {

```

```

    ll x, y;
    ll g = exgcd(a, m, x, y);
    // Ensure the result is positive
    return (x % m + m) % m;
}

/**
 * Bit-reversal permutation.
 * Reorders the elements of 'a' into the order required by iterative NTT.
 */
void bit_rev(vector<int> &a, int n) {
    vector<int> rev(n);
    for (int i = 0; i < n; ++i) {
        rev[i] = rev[i >> 1] >> 1;
        if (i & 1) rev[i] = n >> 1;
    }
    for (int i = 0; i < n; ++i) if (i < rev[i]) swap(a[i], a[rev[i]]);
}

/**
 * Modular exponentiation (Quick Power).
 * Calculates (x^y) % mod.
 */
ll qpow(ll x, ll y, ll mod) {
    ll re = 1;
    for (; y; y >>= 1, x = x * x % mod) if (y & 1) re = re * x % mod;
    return re;
}

/**
 * Number Theoretic Transform (NTT) main function.
 * a The polynomial (vector of coefficients) to transform.
 * mod The NTT-friendly modulus to use.
 * g The primitive root for that modulus.
 * inv 0 for forward transform, 1 for inverse transform.
 */
void ntt(vector<int> &a, int mod, int g, int inv) {
    int n = a.size();
    bit_rev(a, n); // Reorder for iterative butterfly operations

    // Iterative butterfly loops
    for (int len = 2; len <= n; len <= 1) { // len = current subarray size
        // Precalculate the root of unity for this length
        int exp = (mod - 1) / len;
        int wlen = qpow(g, exp, mod);
        if (inv) wlen = modinv(wlen, mod); // Use inverse root for inverse
    }
}
NTT

```

```

        for (int i = 0; i < n; i += len) { // Iterate over all subarrays
            int w = 1; // w = current power of the root of unity
            for (int j = 0; j < len / 2; ++j) {
                // Butterfly operation
                int u = a[i + j];
                int v = (ll)a[i + j + len / 2] * w % mod;
                a[i + j] = (u + v) % mod;
                a[i + j + len / 2] = (u - v + mod) % mod; // (u - v) can be
negative
                w = (ll)w * wlen % mod; // Next power of root
            }
        }

        // Scale by 1/n for inverse transform
        if (inv) {
            ll inv_n = modinv(n, mod);
            for (int &x : a) x = (ll)x * inv_n % mod;
        }
    }

/**
 * Polynomial multiplication under a single NTT-friendly modulus.
 * Calculates (a * b) % mod.
 */
vector<int> poly_mult_mod(const vector<int> &a, const vector<int> &b, int
mod, int g) {
    vector<int> fa(a.begin(), a.end()), fb(b.begin(), b.end());
    int n = 1;
    int len = a.size() + b.size() - 1; // Resulting polynomial degree

    // Pad vectors to the next power of 2
    while (n < len) n <= 1;
    fa.resize(n, 0);
    fb.resize(n, 0);

    // 1. Forward NTT (Coefficient → Point-value form)
    ntt(fa, mod, g, 0);
    ntt(fb, mod, g, 0);

    // 2. Point-wise multiplication (O(N))
    vector<int> fc(n);
    for (int i = 0; i < n; ++i) fc[i] = (ll)fa[i] * fb[i] % mod;

    // 3. Inverse NTT (Point-value → Coefficient form)
    ntt(fc, mod, g, 1);

    // 4. Resize to the actual degree

```

```

        fc.resize(len);
        return fc;
    }

/**
 * Solves a system of 2 congruences using CRT.
 * Finds x such that:
 *  $x \equiv a_1 \pmod{m_1}$ 
 *  $x \equiv a_2 \pmod{m_2}$ 
 * return {x, lcm(m1, m2)}
 */
pair<ll, ll> crt2(ll a1, ll m1, ll a2, ll m2) {
    ll x, y;
    ll g = exgcd(m1, m2, x, y); //  $x*m_1 + y*m_2 = g$ 

    ll lcm = (ll)m1 / g * m2;
    // Calculate the result based on the formula derived from exgcd
    x = (x * ((a2 - a1) / g)) % (m2 / g);
    x = (x + (m2 / g)) % (m2 / g); // Ensure positive
    ll res = (a1 + x * m1) % lcm;
    return {res, lcm};
}

/**
 * Solves a system of 3 congruences using CRT.
 * Finds x such that:
 *  $x \equiv a_1 \pmod{MOD1}$ 
 *  $x \equiv a_2 \pmod{MOD2}$ 
 *  $x \equiv a_3 \pmod{MOD3}$ 
 * return The final result  $x \% MOD$  (our target modulo).
 */
ll crt3(ll a1, ll a2, ll a3) {
    // 1. Solve for (a1, MOD1) and (a2, MOD2) first
    auto [x1, m1] = crt2(a1, MOD1, a2, MOD2);
    // 2. Use that result to solve against (a3, MOD3)
    auto [x2, m2] = crt2(x1, m1, a3, MOD3);
    // 3. Return the final answer modulo the problem's MOD
    return x2 % MOD;
}

/**
 * Polynomial multiplication  $(A * B) \% MOD$ .
 * Uses 3-Mod NTT + CRT to get the result.
 */
vector<int> poly_mult(const vector<int> &a, const vector<int> &b) {
    // 1. Calculate convolution under 3 different moduli
    vector<int> c1 = poly_mult_mod(a, b, MOD1, G1);
    vector<int> c2 = poly_mult_mod(a, b, MOD2, G2);

```



```

vector<int> c3 = poly_mult_mod(a, b, MOD3, G3);

int len = c1.size();
vector<int> res(len);

// 2. Combine results for each coefficient using CRT
for (int i = 0; i < len; ++ i) res[i] = crt3(c1[i], c2[i], c3[i]);
return res;
}

/**
 *Utility function to keep an integer within the [0, MOD-1] range.
 */
void reduce(int &x) {x = (x ≥ MOD ? x - MOD : x);}

/**
 * Polynomial addition/subtraction: A = A + (B * x^shift) * mul.
 * A and B are polynomials represented as vectors.
 */
void add(vector<int> &A, vector<int> &B, int shift = 0, int mul = 1) {
    A.resize(max(A.size(), B.size() + shift));
    for (int i = 0; i < B.size(); ++ i) reduce(A[i + shift] += (mul == 1 ?
B[i] : MOD - B[i]));
}

int main() {
    int n;
    cin >> n;

    // logn = max possible black-height, h = O(log N)
    int logn = log2(n) + 2;
    int ans = 0;

    // Find the smallest power of 2 ≥ n, for polynomial size
    int N = 1;
    while(N < n) N <<= 1;

    // F = Generating Function for Black-rooted trees of black-height h
    // F(x) = B_h(x) = sum(B(h, n) * x^n)
    vector<int> F(N, 0);
    // B_0(x) = 1 (representing a single NULL node at n=0, h=0)
    F[0] = 1;

    // Iterate through black-height h (from i=0 to logn-1)
    for (int i = 0; i < logn; ++ i) {
        // At the start of this loop, F = B_h(x) (where h = i)
        // We want to compute B_{h+1}(x)
    }
}

```

```

// The recurrence relation is:
//  $B_{h+1}(x) = x * (B_h(x))^2 * (1 + x * B_h(x))^2$ 

vector<int> mul1, mul2, mul3;

// 1. mul1 = (B_h(x))^2
// This multiplication is O(N log N) thanks to NTT
mul1 = poly_mult(F, F);

// 2. mul2 = (1 + x * B_h(x))
mul2.push_back(1); // Start with 1
add(mul2, F, 1); // Add F, shifted by 1 (which is x * F)

// 3. mul3 = (1 + x * B_h(x))^2
mul3 = poly_mult(mul2, mul2); // O(N log N)

// Resize to N to keep polynomial sizes consistent
mul1.resize(N);
mul3.resize(N);

// 4. F = (B_h(x))^2 * (1 + x * B_h(x))^2
F = poly_mult(mul1, mul3); // O(N log N)

// 5. F = x * [ (B_h(x))^2 * (1 + x * B_h(x))^2 ]
// This is B_{h+1}(x)
F.insert(F.begin(), 0); // Insert 0 at coeff 0, shifting all others
right
F.resize(N); // Truncate back to size N

// The answer is the sum of B(h, N) for all h > 0.
// F[n] is the coefficient of x^n in B_{h+1}(x), which is B(h+1,
N).
// We add this to our total answer.
reduce(ans += F[n]);
}

cout << ans << "\n";
}

```

Declaration

I hereby declare that all the work done in this project is of our group's independent effort.